

class059 建图、链式前向星、拓扑排序 笔记

视频：算法讲解059【必备】建图、链式前向星、拓扑排序

题目1：三种建图方式

题面

演示三种图的存储方式：邻接矩阵、邻接表、链式前向星。分别用于有向图和无向图带权图的建立和遍历。

核心思路

邻接矩阵：`graph[u][v] = w`。O(1)查询边，O(V²)空间。适合稠密图。

邻接表：每个节点维护一个邻居列表。动态灵活，但遍历缓存不友好。

链式前向星（重点）：静态数组模拟链表，竞赛首选。

```
1  int head[MAXN], nxt[MAXM], to[MAXM], weight[MAXM], cnt;
2  void addEdge(int u, int v, int w) {
3      nxt[cnt] = head[u];
4      to[cnt] = v;
5      weight[cnt] = w;
6      head[u] = cnt++;
7  }
8  // 遍历u的所有邻居：
9  for (int ei = head[u]; ei > 0; ei = nxt[ei]) {
10     int v = to[ei], w = weight[ei];
11 }
```

- 无向图加两次边，MAXM 需设为边数×2
- cnt 从 1 开始（0 作为空的标记）

坑点

- 无向图时 MAXM 要准备 m*2
- 链式前向星 cnt 初始化为 1，head 初始化为 0

代码

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  // 点的最大数量
5  const int MAXN = 11;
6
7  // 边的最大数量
8  // 只有链式前向星方式建图需要这个数量
9  // 注意如果无向图的最大数量是m条边，数量要准备m*2
```

```

10 // 因为一条无向边要加两条有向边
11 const int MAXM = 21;
12
13 // 邻接矩阵方式建图
14 int graph1[MAXN][MAXN];
15
16 // 邻接表方式建图
17 // graph2[i] = {{邻居, 边权}, ...}
18 vector<vector<pair<int, int>>> graph2;
19
20 // 链式前向星方式建图
21 int head[MAXN];
22 int nxt[MAXM];
23 int to[MAXM];
24
25 // 如果边有权重, 那么需要这个数组
26 int weight[MAXM];
27
28 int cnt;
29
30 void build(int n) {
31     // 邻接矩阵清空
32     for (int i = 1; i <= n; i++) {
33         for (int j = 1; j <= n; j++) {
34             graph1[i][j] = 0;
35         }
36     }
37     // 邻接表清空和准备
38     // 下标需要支持1~n, 所以加入n+1个列表, 0下标准备但不用
39     graph2.clear();
40     graph2.resize(n + 1);
41     // 链式前向星清空
42     cnt = 1;
43     for (int i = 1; i <= n; i++) {
44         head[i] = 0;
45     }
46 }
47
48 // 链式前向星加边
49 void addEdge(int u, int v, int w) {
50     // u -> v, 边权重是w
51     nxt[cnt] = head[u];
52     to[cnt] = v;
53     weight[cnt] = w;
54     head[u] = cnt++;
55 }
56
57 // 三种方式建立有向图带权图
58 void directGraph(int n, int edges[][3], int m) {
59     // 邻接矩阵建图
60     for (int i = 0; i < m; i++) {
61         graph1[edges[i][0]][edges[i][1]] = edges[i][2];
62     }

```

```

63 // 邻接表建图
64 for (int i = 0; i < m; i++) {
65     graph2[edges[i][0]].push_back({edges[i][1], edges[i][2]});
66 }
67 // 链式前向星建图
68 for (int i = 0; i < m; i++) {
69     addEdge(edges[i][0], edges[i][1], edges[i][2]);
70 }
71 }
72
73 // 三种方式建立无向图带权图
74 void undirectGraph(int n, int edges[][3], int m) {
75     // 邻接矩阵建图
76     for (int i = 0; i < m; i++) {
77         graph1[edges[i][0]][edges[i][1]] = edges[i][2];
78         graph1[edges[i][1]][edges[i][0]] = edges[i][2];
79     }
80     // 邻接表建图
81     for (int i = 0; i < m; i++) {
82         graph2[edges[i][0]].push_back({edges[i][1], edges[i][2]});
83         graph2[edges[i][1]].push_back({edges[i][0], edges[i][2]});
84     }
85     // 链式前向星建图
86     for (int i = 0; i < m; i++) {
87         addEdge(edges[i][0], edges[i][1], edges[i][2]);
88         addEdge(edges[i][1], edges[i][0], edges[i][2]);
89     }
90 }
91
92 void traversal(int n) {
93     cout << "邻接矩阵遍历 :" << endl;
94     for (int i = 1; i <= n; i++) {
95         for (int j = 1; j <= n; j++) {
96             cout << graph1[i][j] << " ";
97         }
98         cout << endl;
99     }
100     cout << "邻接表遍历 :" << endl;
101     for (int i = 1; i <= n; i++) {
102         cout << i << "(邻居、边权) : ";
103         for (auto& edge : graph2[i]) {
104             cout << "(" << edge.first << ", " << edge.second << ") ";
105         }
106         cout << endl;
107     }
108     cout << "链式前向星 :" << endl;
109     for (int i = 1; i <= n; i++) {
110         cout << i << "(邻居、边权) : ";
111         // 注意这个for循环，链式前向星的方式遍历
112         for (int ei = head[i]; ei > 0; ei = nxt[ei]) {
113             cout << "(" << to[ei] << ", " << weight[ei] << ") ";
114         }
115         cout << endl;

```

```
116     }
117 }
```

题目2：课程表II（拓扑排序模版）

题面

numCourses 门课，prerequisites[i]=[ai,bi] 表示修 ai 前必须先修 bi。返回一个合法的学习顺序，若不可能则返回空。

- LeetCode 210: <https://leetcode.cn/problems/course-schedule-ii/>

核心思路

Kahn算法（BFS拓扑排序）：

1. 建图 + 入度表：b→a（先修课指向后修课），indegree[a]++
2. 入度为 0 的点入队
3. 每次出队一个点，将其邻居入度-1，减到 0 则入队
4. 出队顺序即拓扑序。若出队数 ≠ n，说明有环

坑点

- 边的方向：bi→ai（bi 是 ai 的先修课）
- 队列用数组模拟（不用 STL queue）

代码

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int MAXN = 2005;
5
6  // 拓扑排序，入度表
7  int indegree[MAXN];
8
9  // 拓扑排序，用到队列
10 int queue_[MAXN];
11 int l, r;
12
13 class Solution {
14 public:
15     vector<int> findOrder(int numCourses, vector<vector<int>>& prerequisites) {
16         // 邻接表建图（和Java一样用动态方式）
17         vector<vector<int>> graph(numCourses);
18         // 入度表初始化
19         for (int i = 0; i < numCourses; i++) {
20             indegree[i] = 0;
21         }
22         for (auto& edge : prerequisites) {
```

```

23         graph[edge[1]].push_back(edge[0]);
24         indegree[edge[0]]++;
25     }
26     l = 0;
27     r = 0;
28     for (int i = 0; i < numCourses; i++) {
29         if (indegree[i] == 0) {
30             queue_[r++] = i;
31         }
32     }
33     int cnt = 0;
34     while (l < r) {
35         int cur = queue_[l++];
36         cnt++;
37         for (int next : graph[cur]) {
38             if (--indegree[next] == 0) {
39                 queue_[r++] = next;
40             }
41         }
42     }
43     if (cnt == numCourses) {
44         return vector<int>(queue_, queue_ + numCourses);
45     }
46     return vector<int>();
47 }
48 };

```

题目3-4：拓扑排序（牛客）

题面

给 n 个点 m 条边的有向图，输出拓扑排序结果。若无合法拓扑序输出 -1。

- 牛客：<https://www.nowcoder.com/practice/88f7e156ca7d43a1a535f619cd3f495c>

核心思路

同题目2的 Kahn 算法。提供两个版本：

- 动态版**：每次建图用邻接表
- 静态版**：用链式前向星建图，全局数组复用，更快

坑点

- 点多边多 ($n, m \leq 200000$)，必须用链式前向星防卡空间
- 入度表、队列等全部用全局静态数组

代码 (动态版)

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int MAXN = 200001;
5
6  // 拓扑排序, 用到队列
7  int queue_[MAXN];
8  int l, r;
9
10 // 拓扑排序, 入度表
11 int indegree[MAXN];
12
13 // 收集拓扑排序的结果
14 int ans[MAXN];
15
16 // 邻接表建图
17 int head[MAXN];
18 int nxt[MAXN];
19 int to[MAXN];
20 int graphCnt;
21
22 int n, m;
23
24 void build(int n) {
25     graphCnt = 1;
26     for (int i = 0; i <= n; i++) {
27         head[i] = 0;
28         indegree[i] = 0;
29     }
30 }
31
32 void addEdge(int f, int t) {
33     nxt[graphCnt] = head[f];
34     to[graphCnt] = t;
35     head[f] = graphCnt++;
36 }
37
38 // 有拓扑排序返回true
39 // 没有拓扑排序返回false
40 bool topoSort() {
41     l = r = 0;
42     for (int i = 1; i <= n; i++) {
43         if (indegree[i] == 0) {
44             queue_[r++] = i;
45         }
46     }
47     int fill = 0;
48     while (l < r) {
49         int cur = queue_[l++];
50         ans[fill++] = cur;
```

```

51         // 用链式前向星的方式，遍历cur的相邻节点
52         for (int ei = head[cur]; ei > 0; ei = nxt[ei]) {
53             if (--indegree[to[ei]] == 0) {
54                 queue_[r++] = to[ei];
55             }
56         }
57     }
58     return fill == n;
59 }

```

代码（静态版）

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int MAXN = 200001;
5  const int MAXM = 200001;
6
7  // 建图相关，链式前向星
8  int head[MAXN];
9  int nxt[MAXM];
10 int to[MAXM];
11 int cnt;
12
13 // 拓扑排序，用到队列
14 int queue_[MAXN];
15 int l, r;
16
17 // 拓扑排序，入度表
18 int indegree[MAXN];
19
20 // 收集拓扑排序的结果
21 int ans[MAXN];
22
23 int n, m;
24
25 void build(int n) {
26     cnt = 1;
27     for (int i = 0; i <= n; i++) {
28         head[i] = 0;
29         indegree[i] = 0;
30     }
31 }
32
33 // 用链式前向星建图
34 void addEdge(int f, int t) {
35     nxt[cnt] = head[f];
36     to[cnt] = t;
37     head[f] = cnt++;
38 }
39
40 bool toposort() {

```

```

41     l = r = 0;
42     for (int i = 1; i <= n; i++) {
43         if (indegree[i] == 0) {
44             queue_[r++] = i;
45         }
46     }
47     int fill = 0;
48     while (l < r) {
49         int cur = queue_[l++];
50         ans[fill++] = cur;
51         // 用链式前向星的方式，遍历cur的相邻节点
52         for (int ei = head[cur]; ei != 0; ei = nxt[ei]) {
53             if (--indegree[to[ei]] == 0) {
54                 queue_[r++] = to[ei];
55             }
56         }
57     }
58     return fill == n;
59 }

```

题目5：字典序最小的拓扑排序（洛谷）

题面

在合法的拓扑排序中输出字典序最小的那个。

核心思路

将队列换成**小根堆**：每次从入度为 0 的点中取编号最小的。其他同 Kahn 算法。

- 手写堆：heapInsert + heapify
- push(x), pop(), isEmpty()

坑点

- 不能用优先队列（STL），手写小根堆保持静态风格
- 堆操作注意维护 heapSize

代码

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int MAXN = 100001;
5  const int MAXM = 100001;
6
7  // 建图相关，链式前向星
8  int head[MAXN];
9  int nxt[MAXM];
10 int to[MAXM];

```



```

11  int cnt;
12
13  // 拓扑排序，不用队列，用小根堆，为了得到字典序最小的拓扑排序
14  int heap[MAXN];
15  int heapSize;
16
17  // 拓扑排序，入度表
18  int indegree[MAXN];
19
20  // 收集拓扑排序的结果
21  int ans[MAXN];
22
23  int n, m;
24
25  // 清理之前的脏数据
26  void build(int n) {
27      cnt = 1;
28      heapSize = 0;
29      for (int i = 0; i <= n; i++) {
30          head[i] = 0;
31          indegree[i] = 0;
32      }
33  }
34
35  // 用链式前向星建图
36  void addEdge(int f, int t) {
37      nxt[cnt] = head[f];
38      to[cnt] = t;
39      head[f] = cnt++;
40  }
41
42  // 小根堆里加入数字
43  void push(int num) {
44      int i = heapSize++;
45      heap[i] = num;
46      // heapInsert的过程
47      while (heap[i] < heap[(i - 1) / 2]) {
48          swap(heap[i], heap[(i - 1) / 2]);
49          i = (i - 1) / 2;
50      }
51  }
52
53  // 小根堆里弹出最小值
54  int pop() {
55      int ans = heap[0];
56      heap[0] = heap[--heapSize];
57      // heapify的过程
58      int i = 0;
59      int l = 1;
60      while (l < heapSize) {
61          int best = l + 1 < heapSize && heap[l + 1] < heap[l] ? l + 1 : l;
62          best = heap[best] < heap[i] ? best : i;
63          if (best == i) {

```

```

64         break;
65     }
66     swap(heap[best], heap[i]);
67     i = best;
68     l = i * 2 + 1;
69 }
70 return ans;
71 }
72
73 // 判断小根堆是否为空
74 bool isEmpty() {
75     return heapSize == 0;
76 }
77
78 void topoSort() {
79     for (int i = 1; i <= n; i++) {
80         if (indegree[i] == 0) {
81             push(i);
82         }
83     }
84     int fill = 0;
85     while (!isEmpty()) {
86         int cur = pop();
87         ans[fill++] = cur;
88         // 用链式前向星的方式，遍历cur的相邻节点
89         for (int ei = head[cur]; ei != 0; ei = nxt[ei]) {
90             if (--indegree[to[ei]] == 0) {
91                 push(to[ei]);
92             }
93         }
94     }
95 }

```

题目6：火星词典

题面

给定已按外星语言字典序排好的单词列表，推导字母顺序。若矛盾返回 ""。

- LeetCode 269: <https://leetcode.cn/problems/alien-dictionary/>

核心思路

- 出现过的字符 indegree 初始为 0，未出现为 -1
- 比较相邻单词：找到第一个不同字符 `cur[j] != next[j]`，建立 `cur[j]→next[j]` 的边
- 特殊情况**：若 `cur` 是 `next` 的前缀且更长（如 "abc" "ab"），矛盾返回 ""
- 拓扑排序，看是否所有出现过的字符都被输出

坑点

- indegree 初始化为 -1 标记未出现，出现过的字符才设为 0
- 前缀矛盾检查: `j < cur.length() && j == next.length()`

代码

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  // 入度表, 26种字符
5  int indegree[26];
6
7  // 拓扑排序, 队列
8  int queue_[26];
9  int l, r;
10
11 class Solution {
12 public:
13     string alienOrder(vector<string>& words) {
14         for (int i = 0; i < 26; i++) {
15             indegree[i] = -1;
16         }
17         for (string& w : words) {
18             for (char c : w) {
19                 indegree[c - 'a'] = 0;
20             }
21         }
22         // 'a' -> 0
23         // 'b' -> 1
24         // 'z' -> 25
25         // x -> x - 'a'
26         // 邻接表建图 (和Java一样用动态方式)
27         vector<vector<int>> graph(26);
28         for (int i = 0, j, len; i < (int)words.size() - 1; i++) {
29             string& cur = words[i];
30             string& next = words[i + 1];
31             j = 0;
32             len = min((int)cur.length(), (int)next.length());
33             for (; j < len; j++) {
34                 if (cur[j] != next[j]) {
35                     graph[cur[j] - 'a'].push_back(next[j] - 'a');
36                     indegree[next[j] - 'a']++;
37                     break;
38                 }
39             }
40             if (j < (int)cur.length() && j == (int)next.length()) {
41                 return "";
42             }
43         }
44         l = 0, r = 0;
45         int kinds = 0;
```

```

46         for (int i = 0; i < 26; i++) {
47             if (indegree[i] != -1) {
48                 kinds++;
49             }
50             if (indegree[i] == 0) {
51                 queue_[r++] = i;
52             }
53         }
54         string ans;
55         while (l < r) {
56             int cur = queue_[l++];
57             ans += (char)(cur + 'a');
58             for (int next : graph[cur]) {
59                 if (--indegree[next] == 0) {
60                     queue_[r++] = next;
61                 }
62             }
63         }
64         return (int)ans.length() == kinds ? ans : "";
65     }
66 };

```

题目7：戳印序列

题面

给定印章 stamp 和目标 target，初始 ???。每次盖章必须完全盖在序列内（可覆盖已盖章部分）。返回盖章的最左索引顺序，使得最终得到 target。使用次数 $\leq 10 * n$ 。

- LeetCode 936: <https://leetcode.cn/problems/stamping-the-sequence/>

核心思路

反向拓扑排序：从最终状态反推。

1. 枚举每个可能的盖章位置 i ($0 \leq i \leq n - m$)， $\text{indegree}[i] = m$ 表示还需匹配 m 个字符
2. 比较 $\text{target}[i+j]$ 和 $\text{stamp}[j]$ ：匹配则 $\text{indegree}[i]--$ ；不匹配则建立"错误位置 $\rightarrow i$ "的依赖边
3. $\text{indegree}[i] = 0$ 的位置入队（表示可直接盖章）
4. 队列弹出过程：每次盖章消除一些"错误"，解除依赖，最终得到盖章顺序
5. 结果逆序（因为是反向推导）

坑点

- 依赖方向：错误位置 $j \rightarrow$ 依赖它的盖章起始位置 i
- 最终结果需要逆序
- visited 数组防重复统计同位置错误

代码

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int MAXN = 1001;
5  const int MAXM = 1001;
6
7  // 邻接表建图
8  int head[MAXN];
9  int nxt[MAXN * MAXM];
10 int to[MAXN * MAXM];
11 int graphCnt;
12
13 int indegree[MAXN];
14 int queue_[MAXN];
15 int l, r;
16 bool visited[MAXN];
17 int path[MAXN];
18 int pathSize;
19
20 void build(int n) {
21     graphCnt = 1;
22     for (int i = 0; i <= n; i++) {
23         head[i] = 0;
24     }
25 }
26
27 void addEdge(int f, int t) {
28     nxt[graphCnt] = head[f];
29     to[graphCnt] = t;
30     head[f] = graphCnt++;
31 }
32
33 class Solution {
34 public:
35     vector<int> movesToStamp(string stamp, string target) {
36         char* s = &stamp[0];
37         char* t = &target[0];
38         int m = stamp.length();
39         int n = target.length();
40         // indegree[i]表示以i位置开头盖印章，有多少个字符已经和最终目标一致
41         for (int i = 0; i <= n - m; i++) {
42             indegree[i] = m;
43         }
44         build(n);
45         l = 0, r = 0;
46         // O(n*m)
47         for (int i = 0; i <= n - m; i++) {
48             // i开头....(m个)
49             // i+0 i+1 i+m-1
50             for (int j = 0; j < m; j++) {
```

```

51         if (t[i + j] == s[j]) {
52             if (--indegree[i] == 0) {
53                 queue_[r++] = i;
54             }
55         } else {
56             // i + j
57             // from : 错误的位置
58             // to : i开头的下标
59             addEdge(i + j, i);
60         }
61     }
62 }
63 // 同一个位置取消错误不要重复统计
64 for (int i = 0; i < n; i++) {
65     visited[i] = false;
66 }
67 pathSize = 0;
68 while (l < r) {
69     int cur = queue_[l++];
70     path[pathSize++] = cur;
71     for (int i = 0; i < m; i++) {
72         // cur : 开头位置
73         // cur + 0 cur + 1 cur + 2 ... cur + m - 1
74         if (!visited[cur + i]) {
75             visited[cur + i] = true;
76             for (int ei = head[cur + i]; ei > 0; ei = nxt[ei]) {
77                 int next = to[ei];
78                 if (--indegree[next] == 0) {
79                     queue_[r++] = next;
80                 }
81             }
82         }
83     }
84 }
85 if (pathSize != n - m + 1) {
86     return vector<int>();
87 }
88 // path逆序调整
89 vector<int> result(pathSize);
90 for (int i = 0; i < pathSize; i++) {
91     result[i] = path[pathSize - 1 - i];
92 }
93 return result;
94 }
95 };

```

class060 拓扑排序的扩展技巧 笔记

视频：算法讲解060【必备】拓扑排序的扩展技巧

题目1：最大食物链计数

题面

给定有向无环图， $a \rightarrow b$ 表示 a 被 b 捕食。求从最初级动物到最顶级捕食者的食物链条数，答案对 80112002 取模。

- 洛谷 P4017: <https://www.luogu.com.cn/problem/P4017>

核心思路

拓扑排序 + DP 推送：

`lines[i]`：到达节点 i 的食物链条数

- 入度为 0 的点 `lines[i]=1`（每条初级动物自身是一条链的起点）
- 拓扑排序遍历中，每条边 $u \rightarrow v$ ：`lines[v] = (lines[v] + lines[u]) % MOD`
- `head[u]==0`（无出边）的节点是顶级捕食者，将其 `lines` 累加到答案

坑点

- MOD = 80112002（不是常见的 $1e9+7$ ）
- 答案只累加没有出边的节点（顶级捕食者）

代码

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int MAXN = 5001;
5  const int MAXM = 500001;
6  const int MOD = 80112002;
7
8  // 链式前向星建图
9  int head[MAXN];
10 int nxt[MAXM];
11 int to[MAXM];
12 int cnt;
13
14 // 拓扑排序需要的队列
15 int queue_[MAXN];
16
17 // 拓扑排序需要的入度表
18 int indegree[MAXN];
19
20 // 拓扑排序需要的推送信息
21 int lines[MAXN];
22
23 int n, m;
24
25 void build(int n) {
26     cnt = 1;
27     for (int i = 0; i <= n; i++) {
```

```

28     indegree[i] = 0;
29     lines[i] = 0;
30     head[i] = 0;
31 }
32 }
33
34 void addEdge(int u, int v) {
35     nxt[cnt] = head[u];
36     to[cnt] = v;
37     head[u] = cnt++;
38 }
39
40 int ways() {
41     int l = 0;
42     int r = 0;
43     for (int i = 1; i <= n; i++) {
44         if (indegree[i] == 0) {
45             queue_[r++] = i;
46             lines[i] = 1;
47         }
48     }
49     int ans = 0;
50     while (l < r) {
51         int u = queue_[l++];
52         if (head[u] == 0) {
53             // 当前的u节点不再有后续邻居了
54             ans = (ans + lines[u]) % MOD;
55         } else {
56             for (int ei = head[u], v; ei > 0; ei = nxt[ei]) {
57                 // u -> v
58                 v = to[ei];
59                 lines[v] = (lines[v] + lines[u]) % MOD;
60                 if (--indegree[v] == 0) {
61                     queue_[r++] = v;
62                 }
63             }
64         }
65     }
66     return ans;
67 }

```

题目2：喧闹和富有

题面

richer[i]=[ai,bi] 表示 ai 比 bi 更有钱。求对每个人 x，在所有钱 $\geq x$ 的人中，最安静的人是谁。

- LeetCode 851: <https://leetcode.cn/problems/loud-and-rich/>

核心思路

拓扑排序 + 推送安静值:

- 建图: 更有钱→更没钱 ($a_i \rightarrow b_i$) , 因为信息从富流向穷更自然
- 初始 `ans[i]=i` (自己最安静)
- 拓扑遍历: $cur \rightarrow next$ 时, 若 `quiet[ans[cur]] < quiet[ans[next]]` , 更新 `ans[next] = ans[cur]`
- 这样穷的人会"继承"富人中更安静的选择

坑点

- 边方向: $a_i \rightarrow b_i$ (有钱指向没钱) , 入度为 0 的是最有钱的
- 比较安静值用 `<` (更小=更安静)

代码

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int MAXN = 501;
5
6  // 拓扑排序, 入度表
7  int indegree[MAXN];
8
9  // 拓扑排序, 队列
10 int queue_[MAXN];
11 int l, r;
12
13 class Solution {
14 public:
15     vector<int> loudAndRich(vector<vector<int>>& richer, vector<int>& quiet) {
16         int n = quiet.size();
17         // 邻接表建图 (和Java一样用动态方式)
18         vector<vector<int>> graph(n);
19         for (int i = 0; i < n; i++) {
20             indegree[i] = 0;
21         }
22         for (auto& r : richer) {
23             graph[r[0]].push_back(r[1]);
24             indegree[r[1]]++;
25         }
26         l = 0;
27         r = 0;
28         for (int i = 0; i < n; i++) {
29             if (indegree[i] == 0) {
30                 queue_[r++] = i;
31             }
32         }
33         vector<int> ans(n);
34         for (int i = 0; i < n; i++) {
35             ans[i] = i;
```

```

36     }
37     while (l < r) {
38         int cur = queue_[l++];
39         for (int next : graph[cur]) {
40             if (quiet[ans[cur]] < quiet[ans[next]]) {
41                 ans[next] = ans[cur];
42             }
43             if (--indegree[next] == 0) {
44                 queue_[r++] = next;
45             }
46         }
47     }
48     return ans;
49 }
50 };

```

题目3：并行课程III

题面

n 门课，relations 表示先修关系，time[i] 表示第 $i+1$ 门课耗时。可同时修任意多门课（只要先修课已完成）。求完成所有课的最少月份数。

- LeetCode 2050: <https://leetcode.cn/problems/parallel-courses-iii/>

核心思路

拓扑排序 + 最早完成时间：

- `cost[cur]`：完成 cur 这门课的最早时间
- 初始入度为 0 的课直接入队
- 对于每个 cur: `cost[cur] += time[cur-1]`（加上自己耗时），更新全局 `ans = max(ans, cost[cur])`
- 推送到 next: `cost[next] = max(cost[next], cost[cur])`（next 必须等 cur 完成才能开始）

坑点

- 课程编号从 1 开始，time 索引从 0 开始 → `time[cur-1]`
- cost 推送取 max（所有先修课中最晚完成的那个决定开始时间）

代码

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int MAXN = 50001;
5
6  // 拓扑排序，入度表
7  int indegree[MAXN];
8

```

```

9 // 拓扑排序, 队列
10 int queue_[MAXN];
11 int l, r;
12
13 // cost[i] : 完成课程i的最早时间
14 int cost[MAXN];
15
16 class Solution {
17 public:
18     int minimumTime(int n, vector<vector<int>>& relations, vector<int>& time) {
19         // 点 : 1...n
20         // 邻接表建图 (和Java一样用动态方式)
21         vector<vector<int>> graph(n + 1);
22         for (int i = 0; i <= n; i++) {
23             indegree[i] = 0;
24             cost[i] = 0;
25         }
26         for (auto& edge : relations) {
27             graph[edge[0]].push_back(edge[1]);
28             indegree[edge[1]]++;
29         }
30         l = 0;
31         r = 0;
32         for (int i = 1; i <= n; i++) {
33             if (indegree[i] == 0) {
34                 queue_[r++] = i;
35             }
36         }
37         int ans = 0;
38         while (l < r) {
39             int cur = queue_[l++];
40             // l : time[0]
41             // x : time[x-1]
42             cost[cur] += time[cur - 1];
43             ans = max(ans, cost[cur]);
44             for (int next : graph[cur]) {
45                 cost[next] = max(cost[next], cost[cur]);
46                 if (--indegree[next] == 0) {
47                     queue_[r++] = next;
48                 }
49             }
50         }
51         return ans;
52     }
53 };

```

题目4：参加会议的最多员工数

题面

n 个员工坐圆桌，每人有一个喜欢的人。只有坐在喜欢的人旁边才参会。求最多参会人数。

- LeetCode 2127: <https://leetcode.cn/problems/maximum-employees-to-be-invited-to-a-meeting/>

核心思路

拓扑排序剥洋葱 + 基环树分析：

每个点出度为 1 (favorite[i])，图由若干基环树组成。

1. 拓扑排序去链：从入度为 0 的点开始，用 `deep` 数组记录环外最长链长度
2. 遍历剩余环：
 - o 大小为 2 的环（小环）：两个人都可以带上各自的最长链 → `sum += 2 + deep[a] + deep[b]`
 - o 大小 > 2 的环（大环）：只能取环上的人，不能带链 → `bigRing = max(bigRing, ringSize)`
3. 答案 = max(小环总人数, 最大大环人数)

坑点

- 环 size=2 可以带上两侧链条（因为两人坐一起，链条可以分别靠在两侧）
- 环 size>2 链条无法插入（会破坏环结构）
- 答案取两种可能性的最大值

代码

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int MAXN = 100001;
5
6  int indegree[MAXN];
7  int queue_[MAXN];
8  int l, r;
9  int deep[MAXN];
10
11 class Solution {
12 public:
13     int maximumInvitations(vector<int>& favorite) {
14         // 图：favorite[a] = b : a -> b
15         int n = favorite.size();
16         for (int i = 0; i < n; i++) {
17             indegree[i] = 0;
18         }
19         for (int i = 0; i < n; i++) {
20             indegree[favorite[i]]++;
21         }
22         l = 0;
```

```

23     r = 0;
24     for (int i = 0; i < n; i++) {
25         if (indegree[i] == 0) {
26             queue_[r++] = i;
27         }
28     }
29     // deep[i] : 不包括i在内, i之前的最长链的长度
30     for (int i = 0; i < n; i++) {
31         deep[i] = 0;
32     }
33     while (l < r) {
34         int cur = queue_[l++];
35         int next = favorite[cur];
36         deep[next] = max(deep[next], deep[cur] + 1);
37         if (--indegree[next] == 0) {
38             queue_[r++] = next;
39         }
40     }
41     // 目前图中的点, 不在环上的点, 都删除了! indegree[i] == 0
42     // 可能性1 : 所有小环(中心个数 == 2), 算上中心点 + 延伸点, 总个数
43     int sumOfSmallRings = 0;
44     // 可能性2 : 所有大环(中心个数 > 2), 只算中心点, 最大环的中心点个数
45     int bigRings = 0;
46     for (int i = 0; i < n; i++) {
47         // 只关心的环!
48         if (indegree[i] > 0) {
49             int ringSize = 1;
50             indegree[i] = 0;
51             for (int j = favorite[i]; j != i; j = favorite[j]) {
52                 ringSize++;
53                 indegree[j] = 0;
54             }
55             if (ringSize == 2) {
56                 sumOfSmallRings += 2 + deep[i] + deep[favorite[i]];
57             } else {
58                 bigRings = max(bigRings, ringSize);
59             }
60         }
61     }
62     return max(sumOfSmallRings, bigRings);
63 }
64 };

```

class061 最小生成树 笔记

视频: 算法讲解061【必备】最小生成树

题目1: Kruskal算法

题面

给定 n 个点 m 条边的无向带权连通图，求最小生成树的边权和。若不连通输出 "orz"。

- 洛谷 P3366: <https://www.luogu.com.cn/problem/P3366>

核心思路

Kruskal (贪心+并查集)，复杂度 $O(m \log m)$ ：

1. 所有边按权重升序排序
2. 依次枚举每条边，用并查集判断两端是否已连通
3. 若不连通则合并，累加权重
4. 选够 $n-1$ 条边即完成

坑点

- 边用 struct 存储 (`int edges[MAXM][3]` 无法用 sort)
- 并查集路径压缩: `if (i != father[i]) father[i] = find(father[i]);`

代码

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  // 时间复杂度  $O(m * \log m) + O(n + m)$ 
5
6  const int MAXN = 5001;
7  const int MAXM = 200001;
8
9  int father[MAXN];
10
11 // u, v, w
12 struct Edge {
13     int u, v, w;
14 } edges[MAXM];
15
16 int n, m;
17
18 void build() {
19     for (int i = 1; i <= n; i++) {
20         father[i] = i;
21     }
22 }
23
24 int find(int i) {
25     if (i != father[i]) {
26         father[i] = find(father[i]);
27     }
```

```

28     return father[i];
29 }
30
31 // 如果x和y本来就是一个集合，返回false
32 // 如果x和y不是一个集合，合并之后返回true
33 bool unionSet(int x, int y) {
34     int fx = find(x);
35     int fy = find(y);
36     if (fx != fy) {
37         father[fx] = fy;
38         return true;
39     } else {
40         return false;
41     }
42 }
43
44 // 核心算法（通常在main中调用）：
45 // sort(edges, edges + m, [](const Edge& a, const Edge& b) { return a.w < b.w; });
46 // int ans = 0, edgeCnt = 0;
47 // for (int i = 0; i < m; i++) {
48 //     if (unionSet(edges[i].u, edges[i].v)) {
49 //         edgeCnt++;
50 //         ans += edges[i].w;
51 //     }
52 //     if (edgeCnt == n - 1) break;
53 // }
54 // if (edgeCnt == n - 1) cout << ans; else cout << "orz";

```

题目2-3: Prim算法（动态版/静态版）

题面

同上，求最小生成树。

核心思路

Prim（贪心+堆），复杂度 $O((n+m) \log n)$ ：

1. 从节点 1 出发，将其所有边加入小根堆
2. 每次弹出权重最小的边，若目标节点未访问则加入 MST
3. 将新节点的所有边加入堆

动态版：每次 push 边到堆，可能有重复边（通过 set 去重）。

静态版（优化）：堆中每个节点只保留一条最小边。

- `where[v]` 三态：-1=未入堆，-2=已弹出， ≥ 0 =堆中位置
- `addOrUpdateOrIgnore(ei)`：若节点未入堆则插入，已在堆则更新最小值

坑点

- 静态版中 `swap` 时要同步更新 `where` 数组
- `pop()` 后 `where[u] = -2` 标记已弹出

代码（动态版）

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  // 时间复杂度  $O(n + m) + O(m * \log m)$ 
5
6  const int MAXN = 5001;
7  const int MAXM = 400001;
8
9  // 链式前向星建图
10 int head[MAXN];
11 int nxt[MAXM];
12 int to[MAXM];
13 int weight[MAXM];
14 int cnt;
15
16 // 手写小根堆（边：到达节点，花费）
17 int heap[MAXM][2];
18 int heapSize;
19
20 void addEdge(int u, int v, int w) {
21     nxt[cnt] = head[u];
22     to[cnt] = v;
23     weight[cnt] = w;
24     head[u] = cnt++;
25 }
26
27 void push(int v, int cost) {
28     int i = heapSize++;
29     heap[i][0] = v;
30     heap[i][1] = cost;
31     while (heap[i][1] < heap[(i - 1) / 2][1]) {
32         swap(heap[i][0], heap[(i - 1) / 2][0]);
33         swap(heap[i][1], heap[(i - 1) / 2][1]);
34         i = (i - 1) / 2;
35     }
36 }
37
38 void pop(int& v, int& cost) {
39     v = heap[0][0];
40     cost = heap[0][1];
41     heap[0][0] = heap[--heapSize][0];
42     heap[0][1] = heap[heapSize][1];
43     int i = 0;
44     int l = 1;
45     while (l < heapSize) {
```



```

46     int best = l + 1 < heapSize && heap[l + 1][1] < heap[l][1] ? l + 1 : l;
47     best = heap[best][1] < heap[i][1] ? best : i;
48     if (best == i) break;
49     swap(heap[best][0], heap[i][0]);
50     swap(heap[best][1], heap[i][1]);
51     i = best;
52     l = i * 2 + 1;
53 }
54 }
55
56 bool heapEmpty() {
57     return heapSize == 0;
58 }
59
60 // 核心Prim逻辑:
61 // bool set[MAXN] = {false};
62 // int nodeCnt = 1;
63 // set[1] = true;
64 // for (int ei = head[1]; ei > 0; ei = nxt[ei]) push(to[ei], weight[ei]);
65 // int ans = 0;
66 // while (!heapEmpty()) {
67 //     int next, cost;
68 //     pop(next, cost);
69 //     if (!set[next]) {
70 //         nodeCnt++;
71 //         set[next] = true;
72 //         ans += cost;
73 //         for (int ei = head[next]; ei > 0; ei = nxt[ei]) push(to[ei], weight[ei]);
74 //     }
75 // }
76 // if (nodeCnt == n) 输出ans; else 输出"orz";

```

代码（静态版）

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int MAXN = 5001;
5  const int MAXM = 400001;
6
7  int n, m;
8
9  // 链式前向星建图
10 int head[MAXN];
11 int nxt[MAXM];
12 int to[MAXM];
13 int weight[MAXM];
14 int cnt;
15
16 // 改写的堆结构
17 int heap[MAXN][2];
18

```

```

19 // where[v] = -1, 表示v这个节点, 从来没有进入过堆
20 // where[v] = -2, 表示v这个节点, 已经弹出过了
21 // where[v] = i (>=0), 表示v这个节点, 在堆上的i位置
22 int where[MAXN];
23
24 // 堆的大小
25 int heapSize;
26
27 // 找到的节点个数
28 int nodeCnt;
29
30 void build() {
31     cnt = 1;
32     heapSize = 0;
33     nodeCnt = 0;
34     for (int i = 1; i <= n; i++) {
35         head[i] = 0;
36         where[i] = -1;
37     }
38 }
39
40 void addEdge(int u, int v, int w) {
41     nxt[cnt] = head[u];
42     to[cnt] = v;
43     weight[cnt] = w;
44     head[u] = cnt++;
45 }
46
47 void heapInsert(int i) {
48     while (heap[i][1] < heap[(i - 1) / 2][1]) {
49         swap(heap[i], heap[(i - 1) / 2]);
50         int a = heap[i][0];
51         int b = heap[(i - 1) / 2][0];
52         where[a] = i;
53         where[b] = (i - 1) / 2;
54         i = (i - 1) / 2;
55     }
56 }
57
58 // 当前处理的是编号为ei的边!
59 void addOrUpdateOrIgnore(int ei) {
60     int v = to[ei];
61     int w = weight[ei];
62     // 去往v点, 权重w
63     if (where[v] == -1) {
64         // v这个点, 从来没有进入过堆!
65         heap[heapSize][0] = v;
66         heap[heapSize][1] = w;
67         where[v] = heapSize++;
68         heapInsert(where[v]);
69     } else if (where[v] >= 0) {
70         // v这个点的记录, 在堆上的位置是where[v]
71         heap[where[v]][1] = min(heap[where[v]][1], w);

```

```

72     heapInsert(where[v]);
73 }
74 }
75
76 int u, w;
77
78 // 堆顶的记录: 节点 -> u、到节点的花费 -> w
79 void pop() {
80     u = heap[0][0];
81     w = heap[0][1];
82     swap(heap[0], heap[--heapSize]);
83     if (heapSize > 0) {
84         where[heap[0][0]] = 0;
85     }
86     where[u] = -2;
87     nodeCnt++;
88     // heapify
89     int i = 0;
90     int l = 1;
91     while (l < heapSize) {
92         int best = l + 1 < heapSize && heap[l + 1][1] < heap[l][1] ? l + 1 : l;
93         best = heap[best][1] < heap[i][1] ? best : i;
94         if (best == i) break;
95         swap(heap[best], heap[i]);
96         int a = heap[best][0];
97         int b = heap[i][0];
98         where[a] = best;
99         where[b] = i;
100        i = best;
101        l = i * 2 + 1;
102    }
103 }
104
105 bool isEmpty() {
106     return heapSize == 0;
107 }
108
109 int prim() {
110     // 1节点出发
111     nodeCnt = 1;
112     where[1] = -2;
113     for (int ei = head[1]; ei > 0; ei = nxt[ei]) {
114         addOrUpdateOrIgnore(ei);
115     }
116     int ans = 0;
117     while (!isEmpty()) {
118         pop();
119         ans += w;
120         for (int ei = head[u]; ei > 0; ei = nxt[ei]) {
121             addOrUpdateOrIgnore(ei);
122         }
123     }
124     return ans;

```

题目4：水资源分配优化

题面

n 栋房子，可在房子内建水井（成本 $wells[i]$ ），或铺设管道连接两房（成本 $pipes[j]$ ）。求为所有房子供水的最低总成本。

- LeetCode 1168: <https://leetcode.cn/problems/optimize-water-distribution-in-a-village/>

核心思路

虚拟节点 0 + Kruskal:

关键 trick: 建立一个虚拟节点 0 代表"水源"，在房子 i 建水井等价于连接 $(0, i, wells[i])$ 。问题转化为求包含节点 $0 \sim n$ 的 MST。

1. 将所有 $wells[i]$ 转为边 $(0, i+1, wells[i])$
2. 将所有 $pipes$ 加入边集
3. Kruskal 求 MST

坑点

- 虚拟节点 0 的技巧是关键
- 房子编号从 1 开始， $wells$ 索引从 0 开始

代码

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int MAXN = 10010;
5
6  struct Edge {
7      int u, v, w;
8  } edges[MAXN << 1];
9
10 int edgeCnt;
11
12 int father[MAXN];
13
14 void build(int n) {
15     edgeCnt = 0;
16     for (int i = 0; i <= n; i++) {
17         father[i] = i;
18     }
19 }
20
21 int find(int i) {
```

```

22     if (i != father[i]) {
23         father[i] = find(father[i]);
24     }
25     return father[i];
26 }
27
28 // 如果x和y, 原本是一个集合, 返回false
29 // 如果x和y, 不是一个集合, 合并之后后返回true
30 bool unionSet(int x, int y) {
31     int fx = find(x);
32     int fy = find(y);
33     if (fx != fy) {
34         father[fx] = fy;
35         return true;
36     } else {
37         return false;
38     }
39 }
40
41 class Solution {
42 public:
43     int minCostToSupplyWater(int n, vector<int>& wells, vector<vector<int>>& pipes) {
44         build(n);
45         for (int i = 0; i < n; i++, edgeCnt++) {
46             // wells : 100    30
47             //          0(1)  1(2)
48             edges[edgeCnt].u = 0;
49             edges[edgeCnt].v = i + 1;
50             edges[edgeCnt].w = wells[i];
51         }
52         for (int i = 0; i < (int)pipes.size(); i++, edgeCnt++) {
53             edges[edgeCnt].u = pipes[i][0];
54             edges[edgeCnt].v = pipes[i][1];
55             edges[edgeCnt].w = pipes[i][2];
56         }
57         sort(edges, edges + edgeCnt, [](const Edge& a, const Edge& b) {
58             return a.w < b.w;
59         });
60         int ans = 0;
61         for (int i = 0; i < edgeCnt; i++) {
62             if (unionSet(edges[i].u, edges[i].v)) {
63                 ans += edges[i].w;
64             }
65         }
66         return ans;
67     }
68 };

```

题目5：检查边长度限制的路径是否存在

题面

无向图，多次询问：是否存在 p_j 到 q_j 的路径，路径上每条边权都 $< limit_j$ 。

- LeetCode 1697: <https://leetcode.cn/problems/checking-existence-of-edge-length-limited-paths/>

核心思路

离线查询 + Kruskal 思想：

- 边按权重排序，询问按 $limit$ 排序
- 双指针：对于每个询问，将所有 $w < limit$ 的边加入并查集
- 检查 p 和 q 是否连通 ($isSameSet$)

坑点

- 询问需要记录原始索引，排序后丢失顺序
- 使用 `struct Question {u, v, limit, idx}` 保留原始位置

代码

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int MAXN = 100001;
5  const int MAXM = 100001;
6
7  struct Edge {
8      int u, v, w;
9  };
10
11 struct Question {
12     int u, v, limit, idx;
13 };
14
15 Edge edges_global[MAXM];
16 Question questions[MAXN];
17 int father[MAXN];
18
19 void build(int n) {
20     for (int i = 0; i < n; i++) {
21         father[i] = i;
22     }
23 }
24
25 int find(int i) {
26     if (i != father[i]) {
27         father[i] = find(father[i]);
28     }
```

```

29     return father[i];
30 }
31
32 bool issameSet(int x, int y) {
33     return find(x) == find(y);
34 }
35
36 void unionSet(int x, int y) {
37     father[find(x)] = find(y);
38 }
39
40 class solution {
41 public:
42     vector<bool> distanceLimitedPathsExist(int n, vector<vector<int>>& edgeList,
43     vector<vector<int>>& queries) {
44         int m = edgeList.size();
45         int k = queries.size();
46         // 将edges转为struct数组以便排序
47         for (int i = 0; i < m; i++) {
48             edges_global[i].u = edgeList[i][0];
49             edges_global[i].v = edgeList[i][1];
50             edges_global[i].w = edgeList[i][2];
51         }
52         sort(edges_global, edges_global + m, [](const Edge& a, const Edge& b) {
53             return a.w < b.w;
54         });
55         for (int i = 0; i < k; i++) {
56             questions[i].u = queries[i][0];
57             questions[i].v = queries[i][1];
58             questions[i].limit = queries[i][2];
59             questions[i].idx = i;
60         }
61         sort(questions, questions + k, [](const Question& a, const Question& b) {
62             return a.limit < b.limit;
63         });
64         build(n);
65         vector<bool> ans(k);
66         for (int i = 0, j = 0; i < k; i++) {
67             // i : 问题编号
68             // j : 边的编号
69             for (; j < m && edges_global[j].w < questions[i].limit; j++) {
70                 unionSet(edges_global[j].u, edges_global[j].v);
71             }
72             ans[questions[i].idx] = issameSet(questions[i].u, questions[i].v);
73         }
74         return ans;
75     };

```

题目6：繁忙的都市（洛谷 P2330）

题面

n 个路口 m 条双向路，每条路有分值。要求：

1. 改造的路连通所有路口
2. 改造路尽量少
3. 在上述条件下，分值最大的路分值尽量小

输出：改造了几条路 和 最大分值

核心思路

Kruskal 天然满足：按分值排序后，前 $n-1$ 条能连通所有路口的边即为答案。最后加入的那条边的分值就是最大分值。

坑点

- 分值更新用 `ans = max(ans, edges[i].w)` 而非累加
- 输出 `n-1` 和 `ans` ($n-1$ 条边 + 最后加入边的分值)

代码

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int MAXN = 301;
5  const int MAXM = 8001;
6
7  int father[MAXN];
8
9  struct Edge {
10     int u, v, w;
11 } edges[MAXM];
12
13 int n, m;
14
15 void build() {
16     for (int i = 1; i <= n; i++) {
17         father[i] = i;
18     }
19 }
20
21 int find(int i) {
22     if (i != father[i]) {
23         father[i] = find(father[i]);
24     }
25     return father[i];
26 }
27
28 // 如果x和y本来就是一个集合，返回false
```



```

29 // 如果x和y不是一个集合，合并之后返回true
30 bool unionSet(int x, int y) {
31     int fx = find(x);
32     int fy = find(y);
33     if (fx != fy) {
34         father[fx] = fy;
35         return true;
36     } else {
37         return false;
38     }
39 }
40
41 // 核心算法：
42 // sort(edges, edges + m, [](const Edge& a, const Edge& b) { return a.w < b.w; });
43 // int ans = 0, edgeCnt = 0;
44 // for (int i = 0; i < m; i++) {
45 //     if (unionSet(edges[i].u, edges[i].v)) {
46 //         edgeCnt++;
47 //         ans = max(ans, edges[i].w);
48 //     }
49 //     if (edgeCnt == n - 1) break;
50 // }
51 // cout << n - 1 << ' ' << ans;

```

class062 宽度优先遍历及其扩展 笔记

视频：算法讲解062【必备】宽度优先遍历及其扩展

题目1：地图分析

题面

$n \times n$ 的网格，0 代表海洋，1 代表陆地。找出一个海洋单元格，使其到最近陆地单元格的曼哈顿距离最大。若全陆地或全海洋返回 -1。

- LeetCode 1162: <https://leetcode.cn/problems/as-far-from-land-as-possible/>

核心思路

多源 BFS：把所有陆地同时入队作为第 0 层，逐层向外扩散。当 BFS 结束时，记录的层数减 1 就是最大距离。

- 陆地入队 visited=true，同时统计海洋数 seas
- 如果 seas==0 或 seas==n*m，直接返回 -1
- 每次处理整层 (size = r-1)，level++ 后扩散四个方向

坑点

- 多源 BFS 初始要把所有陆地一起入队，而非只入一个
- 返回 `level - 1` (因为最后一层是只入队但没扩散的)

代码

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int MAXN = 101;
5  const int MAXM = 101;
6
7  int queue_[MAXN * MAXM][2];
8  int l, r;
9  bool visited[MAXN][MAXM];
10
11 // 0:上, 1:右, 2:下, 3:左
12 int move_[] = { -1, 0, 1, 0, -1 };
13
14 class Solution {
15 public:
16     int maxDistance(vector<vector<int>>& grid) {
17         l = r = 0;
18         int n = grid.size();
19         int m = grid[0].size();
20         int seas = 0;
21         for (int i = 0; i < n; i++) {
22             for (int j = 0; j < m; j++) {
23                 if (grid[i][j] == 1) {
24                     visited[i][j] = true;
25                     queue_[r][0] = i;
26                     queue_[r++][1] = j;
27                 } else {
28                     visited[i][j] = false;
29                     seas++;
30                 }
31             }
32         }
33         if (seas == 0 || seas == n * m) {
34             return -1;
35         }
36         int level = 0;
37         while (l < r) {
38             level++;
39             int size = r - l;
40             for (int k = 0, x, y, nx, ny; k < size; k++) {
41                 x = queue_[l][0];
42                 y = queue_[l++][1];
43                 for (int i = 0; i < 4; i++) {
44                     nx = x + move_[i];
45                     ny = y + move_[i + 1];
```

```

46         if (nx >= 0 && nx < n && ny >= 0 && ny < m && !visited[nx][ny]) {
47             visited[nx][ny] = true;
48             queue_[r][0] = nx;
49             queue_[r++][1] = ny;
50         }
51     }
52 }
53 }
54 return level - 1;
55 }
56 };

```

题目2：贴纸拼词

题面

n 种贴纸，每种无限量，每个贴纸有一个小写英文单词。从贴纸中切割字母拼出 target，求最少贴纸数。

- LeetCode 691: <https://leetcode.cn/problems/stickers-to-spell-word/>

核心思路

BFS + 剪枝：

- 先将所有贴纸和 target 按字母排序，转成有序形式
- 建图：每种贴纸按首字母分类到 `graph[首字母]`，且去重（同一贴纸只加一次）
- BFS 从头开始：每次取当前字符串的首字母，只尝试对应 `graph` 里的贴纸
- `next(t, s)`：用贴纸 s 去消除 t 中的字符，返回剩余字符串
- 剩余为空→成功返回 level；否则检查 visited，未见过则入队

坑点

- 排序是关键优化：让相同前缀的 target 能被哈希去重
- `next()` 中的双指针合并类似归并过程

代码

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int MAXN = 401;
5  string queue_[MAXN];
6  int l, r;
7  vector<vector<string>> graph(26);
8  unordered_set<string> visited;
9
10 string sortStr(const string& str) {
11     string s = str;
12     sort(s.begin(), s.end());

```

```

13     return s;
14 }
15
16 string nextStr(const string& t, const string& s) {
17     string builder;
18     int i = 0, j = 0;
19     while (i < (int)t.length()) {
20         if (j == (int)s.length()) builder += t[i++];
21         else if (t[i] < s[j]) builder += t[i++];
22         else if (t[i] > s[j]) j++;
23         else { i++; j++; }
24     }
25     return builder;
26 }
27
28 class Solution {
29 public:
30     int minStickers(vector<string>& stickers, string target) {
31         for (int i = 0; i < 26; i++) graph[i].clear();
32         visited.clear();
33         for (string& str : stickers) {
34             str = sortStr(str);
35             for (int i = 0; i < (int)str.length(); i++) {
36                 if (i == 0 || str[i] != str[i - 1]) {
37                     graph[str[i] - 'a'].push_back(str);
38                 }
39             }
40         }
41         target = sortStr(target);
42         visited.insert(target);
43         l = r = 0;
44         queue_[r++] = target;
45         int level = 1;
46         while (l < r) {
47             int size = r - l;
48             for (int i = 0; i < size; i++) {
49                 string cur = queue_[l++];
50                 for (string& s : graph[cur[0] - 'a']) {
51                     string next = nextStr(cur, s);
52                     if (next == "") return level;
53                     else if (visited.find(next) == visited.end()) {
54                         visited.insert(next);
55                         queue_[r++] = next;
56                     }
57                 }
58             }
59             level++;
60         }
61         return -1;
62     }
63 };

```

题目3：到达角落需要移除障碍物的最小数目

题面

m×n 网格，0 表示空格，1 表示可移除的障碍。从左上走到右下，求最少移除障碍数。

- LeetCode 2290: <https://leetcode.cn/problems/minimum-obstacle-removal-to-reach-corner/>

核心思路

01-BFS (双端队列)：每条边权为 0 或 1（走到空格代价 0，走到障碍代价 1），用双端队列求最短路。

- 代价为 0 → 队首插入 (`--h`)
- 代价为 1 → 队尾插入 (`++t`)
- 第一次扩展到右下角即可返回（因为 01-BFS 保证第一次遇到即最短路）

坑点

- 双端队列用数组模拟（全局静态数组 + h/t 指针）
- distance 初始化为 `INT_MAX`

代码

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int MAXN = 100001;
5  int deque_[MAXN][2];
6  int h, t;
7  int move_[] = { -1, 0, 1, 0, -1 };
8
9  class Solution {
10 public:
11     int minimumObstacles(vector<vector<int>>& grid) {
12         int m = grid.size(), n = grid[0].size();
13         vector<vector<int>> distance(m, vector<int>(n, INT_MAX));
14         h = t = MAXN / 2;
15         deque_[h][0] = 0; deque_[h][1] = 0;
16         distance[0][0] = 0;
17         while (h <= t) {
18             int x = deque_[h][0], y = deque_[h][1];
19             if (x == m - 1 && y == n - 1) return distance[x][y];
20             for (int i = 0; i < 4; i++) {
21                 int nx = x + move_[i], ny = y + move_[i + 1];
22                 if (nx >= 0 && nx < m && ny >= 0 && ny < n
23                     && distance[x][y] + grid[nx][ny] < distance[nx][ny]) {
24                     distance[nx][ny] = distance[x][y] + grid[nx][ny];
25                     if (grid[nx][ny] == 0) {
26                         deque_[--h][0] = nx; deque_[h][1] = ny;
27                     } else {
28                         deque_[++t][0] = nx; deque_[t][1] = ny;
```

```

29         }
30     }
31 }
32 }
33 return -1;
34 }
35 };

```

题目4：使网格图至少有一条有效路径的最小代价

题面

$m \times n$ 网格，每格数字表示方向（1→右, 2→左, 3→下, 4→上）。花费 1 可修改一格方向（仅一次）。求从 (0,0) 到右下角的最小修改代价。

- LeetCode 1368: <https://leetcode.cn/problems/minimum-cost-to-make-at-least-one-valid-path-in-a-grid/>

核心思路

01-BFS：顺着原方向走代价 0（队首），改方向走代价 1（队尾）。

- `weight = grid[x][y] != i ? 1 : 0`
- 每个格子的 4 个方向都尝试，weight 为 0 或 1

坑点

- `move_` 数组下标从 1 开始（1=右, 2=左, 3=下, 4=上），与 grid 值对应
- 和题目 3 一样用双端队列模拟

代码

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int MAXN = 10001;
5  int deque_[MAXN][2];
6  int h, t;
7  int move_[5][2] = { {}, { 0, 1 }, { 0, -1 }, { 1, 0 }, { -1, 0 } };
8
9  class Solution {
10 public:
11     int minCost(vector<vector<int>>& grid) {
12         int m = grid.size(), n = grid[0].size();
13         vector<vector<int>> distance(m, vector<int>(n, INT_MAX));
14         h = t = MAXN / 2;
15         deque_[h][0] = 0; deque_[h][1] = 0;
16         distance[0][0] = 0;
17         while (h <= t) {
18             int x = deque_[h][0], y = deque_[h++][1];
19             if (x == m - 1 && y == n - 1) return distance[x][y];
20             for (int i = 1; i <= 4; i++) {

```

```

21         int nx = x + move_[i][0], ny = y + move_[i][1];
22         int weight = grid[x][y] != i ? 1 : 0;
23         if (nx >= 0 && nx < m && ny >= 0 && ny < n
24             && distance[x][y] + weight < distance[nx][ny]) {
25             distance[nx][ny] = distance[x][y] + weight;
26             if (weight == 0) {
27                 deque_ [--h][0] = nx; deque_[h][1] = ny;
28             } else {
29                 deque_ [ ++t][0] = nx; deque_[t][1] = ny;
30             }
31         }
32     }
33 }
34 return -1;
35 }
36 };

```

题目5：二维接雨水

题面

$m \times n$ 非负高度矩阵，计算能接多少体积雨水。类似一维接雨水但扩展到二维。

- LeetCode 407: <https://leetcode.cn/problems/trapping-rain-water-ii/>

核心思路

最小堆 + BFS：类似 Dijkstra 的思想，从边界开始“注水”。

- 初始将四条边界全部入小根堆（{行，列，水线高度}）
- 每次弹出堆顶（水线最低的格子），水量 += 水线 - 当前高度
- 向四个方向扩展：入堆的“水线”为 $\max(\text{邻居高度}, \text{当前水线})$ ——水不能低于自身高度
- 手写小根堆加速

坑点

- 手写堆需要维护 `heapInsert` 和 `heapify`
- 水线 = $\max(\text{height}[\text{nr}][\text{nc}], w)$ ：水不能低于当前格子高度

代码

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int MAXN = 201;
5  const int MAXM = 201;
6  int move_[] = { -1, 0, 1, 0, -1 };
7
8  int heapSize;
9  int heap[MAXN * MAXM][3];

```

```

10
11 void push(int r, int c, int w) {
12     int i = heapSize++;
13     heap[i][0] = r; heap[i][1] = c; heap[i][2] = w;
14     while (heap[i][2] < heap[(i - 1) / 2][2]) {
15         swap(heap[i][0], heap[(i - 1) / 2][0]);
16         swap(heap[i][1], heap[(i - 1) / 2][1]);
17         swap(heap[i][2], heap[(i - 1) / 2][2]);
18         i = (i - 1) / 2;
19     }
20 }
21
22 void pop(int& r, int& c, int& w) {
23     r = heap[0][0]; c = heap[0][1]; w = heap[0][2];
24     heap[0][0] = heap[--heapSize][0];
25     heap[0][1] = heap[heapSize][1];
26     heap[0][2] = heap[heapSize][2];
27     int i = 0, l = 1;
28     while (l < heapSize) {
29         int best = l + 1 < heapSize && heap[l + 1][2] < heap[l][2] ? l + 1 : l;
30         best = heap[best][2] < heap[i][2] ? best : i;
31         if (best == i) break;
32         swap(heap[best][0], heap[i][0]);
33         swap(heap[best][1], heap[i][1]);
34         swap(heap[best][2], heap[i][2]);
35         i = best; l = i * 2 + 1;
36     }
37 }
38
39 class Solution {
40 public:
41     int trapRainWater(vector<vector<int>>& height) {
42         int n = height.size(), m = height[0].size();
43         heapSize = 0;
44         vector<vector<bool>> visited(n, vector<bool>(m, false));
45         for (int i = 0; i < n; i++)
46             for (int j = 0; j < m; j++)
47                 if (i == 0 || i == n - 1 || j == 0 || j == m - 1) {
48                     push(i, j, height[i][j]);
49                     visited[i][j] = true;
50                 }
51         int ans = 0;
52         while (heapSize) {
53             int r, c, w;
54             pop(r, c, w);
55             ans += w - height[r][c];
56             for (int i = 0, nr, nc; i < 4; i++) {
57                 nr = r + move_[i]; nc = c + move_[i + 1];
58                 if (nr >= 0 && nr < n && nc >= 0 && nc < m && !visited[nr][nc]) {
59                     push(nr, nc, max(height[nr][nc], w));
60                     visited[nr][nc] = true;
61                 }
62             }

```



```
63     }
64     return ans;
65 }
66 };
```

题目6：单词接龙 II

题面

从 beginWord 变到 endWord，每次变一个字母，中间词必须在 wordList 中。返回所有**最短**转换序列。

- LeetCode 126: <https://leetcode.cn/problems/word-ladder-ii/>

核心思路

BFS 建反向图 + DFS 收集路径：

1. BFS 从 begin 开始逐层扩散：每层把词从 dict 中删掉（去重），每个位置 a~z 替换看是否在 dict 中
2. 在 BFS 过程中**建反向图**：graph[新词].push_back(当前词)，记录"从哪来的"
3. BFS 遇到 end 时立即停止整层处理（因为要所有最短路径）
4. DFS 从 end 反向走向 begin，用 list 收集路径并翻转

坑点

- BFS 必须整层处理完才检查是否找到 end（因为同一层可能有多个路径到达 end）
- 每层开始前 dict.removeAll(curLevel) 而非边扩边删（保证同层不同词可互为邻居）
- path.push_front + pop_front 实现路径回溯

代码

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  unordered_set<string> dict;
5  unordered_set<string> curLevel, nextLevel;
6  unordered_map<string, vector<string>> graph_;
7  list<string> path_;
8  vector<vector<string>> ans_;
9
10 void build_(vector<string>& wordList) {
11     dict = unordered_set<string>(wordList.begin(), wordList.end());
12     graph_.clear(); ans_.clear();
13     curLevel.clear(); nextLevel.clear();
14 }
15
16 bool bfs_(const string& begin, const string& end) {
17     bool find = false;
18     curLevel.insert(begin);
19     while (!curLevel.empty()) {
```

```

20     for (const string& word : curLevel) dict.erase(word);
21     for (const string& word : curLevel) {
22         string w = word;
23         for (int i = 0; i < (int)w.length(); i++) {
24             char old = w[i];
25             for (char ch = 'a'; ch <= 'z'; ch++) {
26                 w[i] = ch;
27                 if (dict.count(w) && w != word) {
28                     if (w == end) find = true;
29                     graph_[w].push_back(word);
30                     nextLevel.insert(w);
31                 }
32             }
33             w[i] = old;
34         }
35     }
36     if (find) return true;
37     curLevel.clear(); swap(curLevel, nextLevel);
38 }
39 return false;
40 }
41
42 void dfs_(const string& word, const string& aim) {
43     path_.push_front(word);
44     if (word == aim) ans_.push_back(vector<string>(path_.begin(), path_.end()));
45     else if (graph_.count(word))
46         for (const string& next : graph_[word]) dfs_(next, aim);
47     path_.pop_front();
48 }
49
50 class Solution {
51 public:
52     vector<vector<string>> findLadders(string beginword, string endword,
53     vector<string>& wordList) {
54         build_(wordList);
55         if (!dict.count(endword)) return ans_;
56         if (bfs_(beginword, endword)) dfs_(endword, beginword);
57         return ans_;
58     };

```

class063 双向广搜 笔记

视频：算法讲解063【必备】双向广搜

用途概述

双向广搜有两大用途：

- **用途一（小优化）**：BFS 从两端展开，哪侧数量少就从哪侧扩，在中间相遇即得最短距离
- **用途二（本体）**：全量暴力展开不可行（如 2^{40} ），但半量展开可行（ 2^{20} ），左右各自展开再整合，用双

题目1：单词接龙（双向广搜用途一）

题面

从 beginWord 变到 endWord，每次变一个字母，中间词必须在 wordList 中。求最短转换序列的单词数。

- LeetCode 127: <https://leetcode.cn/problems/word-ladder/>

核心思路

双向 BFS（哪侧小从哪侧扩）：

- 维护三个集合：smallLevel（数量小的一侧）、bigLevel（数量大的一侧）、nextLevel
- 每次从 small 侧扩展：每个单词每位 a~z 试一遍
- 扩展出的单词如果在 bigLevel 中→相遇，返回 len
- 如果在 dict 中→加入 nextLevel，并从 dict 删掉（去重）
- 每次比较 nextLevel.size() 和 bigLevel.size()，小的一方做下一轮的 small

坑点

- 扩展出的单词必须在 dict 中有才入队，且立即从 dict 删除防止重复
- 交换逻辑：小的做 small，大的做 big，next 始终是"从小侧扩出的下一层"

代码

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class Solution {
5  public:
6      int ladderLength(string begin, string end, vector<string>& wordList) {
7          unordered_set<string> dict(wordList.begin(), wordList.end());
8          if (!dict.count(end)) return 0;
9          unordered_set<string> smallLevel, bigLevel, nextLevel;
10         smallLevel.insert(begin);
11         bigLevel.insert(end);
12         for (int len = 2; !smallLevel.empty(); len++) {
13             for (const string& w : smallLevel) {
14                 string word = w;
15                 for (int j = 0; j < (int)word.length(); j++) {
16                     char old = word[j];
17                     for (char change = 'a'; change <= 'z'; change++) {
18                         if (change != old) {
19                             word[j] = change;
20                             if (bigLevel.count(word)) return len;
21                             if (dict.count(word)) {
22                                 dict.erase(word);
```

```

23         nextLevel.insert(word);
24     }
25 }
26 }
27     word[j] = old;
28 }
29 }
30     if (nextLevel.size() <= bigLevel.size()) {
31         swap(smallLevel, nextLevel);
32     } else {
33         smallLevel.clear();
34         swap(smallLevel, bigLevel);
35         swap(bigLevel, nextLevel);
36     }
37     nextLevel.clear();
38 }
39 return 0;
40 }
41 };

```

题目2：零食问题 / 世界冰球锦标赛（双向广搜用途二）

题面

n 袋零食 ($n \leq 40$)，每袋体积 $v[i]$ ($v[i] \leq 10^9$)，背包容量 w ($w \leq 2 \times 10^9$)。求总体积**不超过** w 的方案数（不拿也算一种）。答案不取模。

- 牛客 / 洛谷 P4799: <https://www.luogu.com.cn/problem/P4799>

核心思路

半量展开 + 双指针整合：

- 全量展开： $2^{40} \approx 10^{12}$ ，不可行
- 分两半：左半 $2^{20} \approx 10^6$ ，右半 $2^{20} \approx 10^6$ ，各自暴力递归收集所有 $\leq w$ 的累加和
- 左右排序后双指针： i 从 $lsum$ 末尾往左， j 从 $rsum$ 开头往右
 - 固定 $lsum[i]$ ，while $lsum[i] + rsum[j] \leq w$: $j++$
 - $ans += j$ （右侧有 j 个方案能和 $lsum[i]$ 组合）
- 复杂度： $O(2^{n/2} \times n)$ 可行

坑点

- 数值巨大 ($v[i]$ 和 w 都可达 2×10^9)，用 `long long`
- 递归展开过程中，累加和超过 w 直接剪枝返回
- 双指针不回退： i 递减时 j 只增不减

代码

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int MAXN = 40;
5  const int MAXM = 1 << 20;
6
7  long long arr[MAXN];
8  long long lsum[MAXM];
9  long long rsum[MAXM];
10 int n;
11 long long w;
12
13 // arr[i..e-1]范围上展开，到达e就停止
14 // 返回值 : ans数组填到了什么位置!
15 int f(int i, int e, long long s, long long w, long long ans[], int j) {
16     if (s > w) return j;
17     if (i == e) { ans[j++] = s; }
18     else {
19         j = f(i + 1, e, s, w, ans, j); // 不要arr[i]
20         j = f(i + 1, e, s + arr[i], w, ans, j); // 要arr[i]
21     }
22     return j;
23 }
24
25 long long compute() {
26     int lsize = f(0, n >> 1, 0, w, lsum, 0);
27     int rsize = f(n >> 1, n, 0, w, rsum, 0);
28     sort(lsum, lsum + lsize);
29     sort(rsum, rsum + rsize);
30     long long ans = 0;
31     for (int i = lsize - 1, j = 0; i >= 0; i--) {
32         while (j < rsize && lsum[i] + rsum[j] <= w) j++;
33         ans += j;
34     }
35     return ans;
36 }
37
38 int main() {
39     ios::sync_with_stdio(false);
40     cin.tie(0);
41     while (cin >> n >> w) {
42         for (int i = 0; i < n; i++) cin >> arr[i];
43         cout << compute() << '\n';
44     }
45     return 0;
46 }
```

题目3：最接近目标值的子序列和（双向广搜用途二）

题面

数组长度 ≤ 40 ，数值 $\pm 10^7$ 。选一个子序列使累加和与 goal 的绝对差最小，返回最小差值。

- LeetCode 1755: <https://leetcode.cn/problems/closest-subsequence-sum/>

核心思路

半量展开 + 剪枝 + 双指针：

- 全量展开 2^{40} 不行，拆左右各 2^{20}
- 剪枝：先算所有正数和 max、所有负数和 min：
 - 若 `max < goal` → 直接返回 `|max-goal|`
 - 若 `min > goal` → 直接返回 `|min-goal|`
- 排序后分组展开（常数优化）：相同的数字按“选 0/1/2/... 个”一组处理，减少递归层数
- 左右排序后双指针：`i` 从左往右、`j` 从右往左，每步看 `j-1` 是否缩小差值，是则滑

坑点

- 排序原数组后分组展开能大幅加速（避免重复递归相同值）
- 双指针不回退：`i` 递增时 `j` 只减不增
- 需要先判断 max/min 与 goal 的关系做剪枝

代码

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int MAXN = 1 << 20;
5  int lsum[MAXN], rsum[MAXN], fill_;
6
7  void collect(int nums[], int i, int e, int s, int sum[]) {
8      if (i == e) { sum[fill_++] = s; }
9      else {
10         int j = i + 1;
11         while (j < e && nums[j] == nums[i]) j++;
12         for (int k = 0; k <= j - i; k++) {
13             collect(nums, j, e, s + k * nums[i], sum);
14         }
15     }
16 }
17
18 class Solution {
19 public:
20     int minAbsDifference(vector<int>& nums, int goal) {
21         int n = nums.size();
22         long long minSum = 0, maxSum = 0;
```

```

23     for (int i = 0; i < n; i++) {
24         if (nums[i] >= 0) maxSum += nums[i];
25         else minSum += nums[i];
26     }
27     if (maxSum < goal) return (int)abs(maxSum - goal);
28     if (minSum > goal) return (int)abs(minSum - goal);
29
30     sort(nums.begin(), nums.end());
31     static int arr[40];
32     for (int i = 0; i < n; i++) arr[i] = nums[i];
33
34     fill_ = 0;
35     collect(arr, 0, n >> 1, 0, lsum);
36     int lsize = fill_;
37     fill_ = 0;
38     collect(arr, n >> 1, n, 0, rsum);
39     int rsize = fill_;
40     sort(lsum, lsum + lsize);
41     sort(rsum, rsum + rsize);
42
43     int ans = abs(goal);
44     for (int i = 0, j = rsize - 1; i < lsize; i++) {
45         while (j > 0 && abs(goal - lsum[i] - rsum[j - 1])
46             <= abs(goal - lsum[i] - rsum[j])) j--;
47         ans = min(ans, abs(goal - lsum[i] - rsum[j]));
48     }
49     return ans;
50 }
51 };

```